

CMDA-4654
Group Exercise 1B

Sophia Gavasheli and Jacob Carlson

October 06, 2026

1. Reintroduction

We are Sophia Gavasheli and Jacob Carlson, and we are group 17. Our chosen domain is the sports category.

2. Executive Summary and Research Questions

kNN Regression

We are using a kNN regression model to predict the runs a team scored in any given game. Runs will be predicted by the following variables.

Variable	Description
AB	At Bats
1b	Singles
2B	Doubles
3B	Triples
HR	Home Runs
SB	Stolen Bases
CS	Caught Stealing
BB	Base on Balls
SO	Strikeouts
HBP	Hit by Pitch
SH	Sacrifice Hits
SF	Sacrifice Flies

The data frame that these variables were taken from was not used in part A of our exercise, but is still part of the larger data set we pulled from. Using runs as a metric was an explored idea in part A, but was not used in our final report, so we decided to further explore it here, since it looked like it could have a relationship with the other variables like home runs and at bats.

kNN Classification and Naive Bayes

For our kNN Classification and Naive Bayes models, we chose to predict a player's position using the following offensive stats:

Variable	Description
BA	Batting Average
HRA	Home Runs Average
BBA	Base on Balls Average
SOA	Strikeout Average

We standardized this data to ensure kNN and Naive Bayes were using the same inputs. We actually spent a lot of our part A analysis on position metric, so exploring further with the two models was a natural part B.

In part A, we used the data frames `fieldingpost.csv` and `batting.csv`. Fielding-post only included post season stats(playoffs), and wasn't big enough for our models, so we switched to `fielding.csv`, which contains the entire seasons. We also used `Teams.csv` for our kNN regression, as it contained all of the data used in our model with minimal cleaning and merging between data sets.

3. Methodology & Modeling Setup

Data Preprocessing

All of our predictors were numerical, so there was no need to handle categorical leveling or do any smoothing to our data. We did a 70/30 training-test split since the data sets were very large, and we preferred to keep the majority in our training.

kNN regression

For our kNN regression, `Teams.csv` includes a great many more predictors than what was used. We built the data frame `team_clean`, which keeps only the American and National leagues, removes `NaN` rows, and takes in only our predictors, `TeamId`, `yearID`, and our prediction variable `Runs`. We also combined the HBP and BB predictors into one, as they provide the same outcome in a game. We also divided all of our predictors by games to establish the per game basis for our prediction.

We standardized our data set under the basic z-score scaling. As mentioned above, the predictor `HBPxBB` was a sum of two predictors, and was unfairly weighted against our other data. Furthermore, given the nature of the game, some of our predictors had a reported 0 weight on our distance, so standardizing was necessary.

The table below shows why scaling matters: for two team-seasons, the unscaled distance is dominated by a few large-valued predictors, while after z-scoring each predictor contributes a more balanced share.

Table 1: Share of squared Euclidean distance contributed by each predictor (first two games) in the kNN Regression Dataset.

	X1B	X2B	X3B	HR	SO	HBPxBB	SB	SF	CS	AB
Raw (share of squared distance, pct)	8.2	0	0.0	0.4	48.6	42.1	0.7	0.0	0.0	0
Z-scored (share of squared distance, pct)	10.8	0	0.1	1.8	9.1	71.6	4.0	1.8	0.7	0

kNN and Naive Bayes Classification

For our kNN classification and Naive Bayes models, we first created `primary_pos`, which read through the player’s data, found their most played position, and changed their other reported positions to the highest one. We then combined `primary_pos` with batting, based on their `playerId` and the `yearID`, so that we could compare the positions to their offensive stats. We also took only the American and National league players, following our analysis in part A.

We wanted to limit classification responses, so we combined the infield positions (1B,2B,3B, SS) into an `Infield` category.

In baseball, different positions get different amounts of playtime. Furthermore, because we combined four possible positions into one, our data was even more skewed, and in the case of kNN classification, our distances would be poorly weighted. We therefore decided to scale all of our predictors based on At bats, providing the predictors described above (BA, HRA, BBA, SOA). While the standardizing would have less affect on our Naive Bayes model, since our goal is to compare it with our classification, we chose to scale the predictors before running either model.

Table 2: Share of squared Euclidean distance contributed by each predictor (first two team-seasons) in the Classification Dataset.

	BA	HRA	BBA	SOA
Raw (share of squared distance, pct)	51.7	13	20.8	14.5
Z-scored (share of squared distance, pct)	40.3	44	13.0	2.8

4. Analysis, Tuning & Results

kNN Regression

Tuning k

We tuned k from $k = 1, \dots, 51$ using 10-fold CV on the training data only. We evaluated candidate k values with Root Mean Squared (RMSE) error which is more sensitive to large errors.

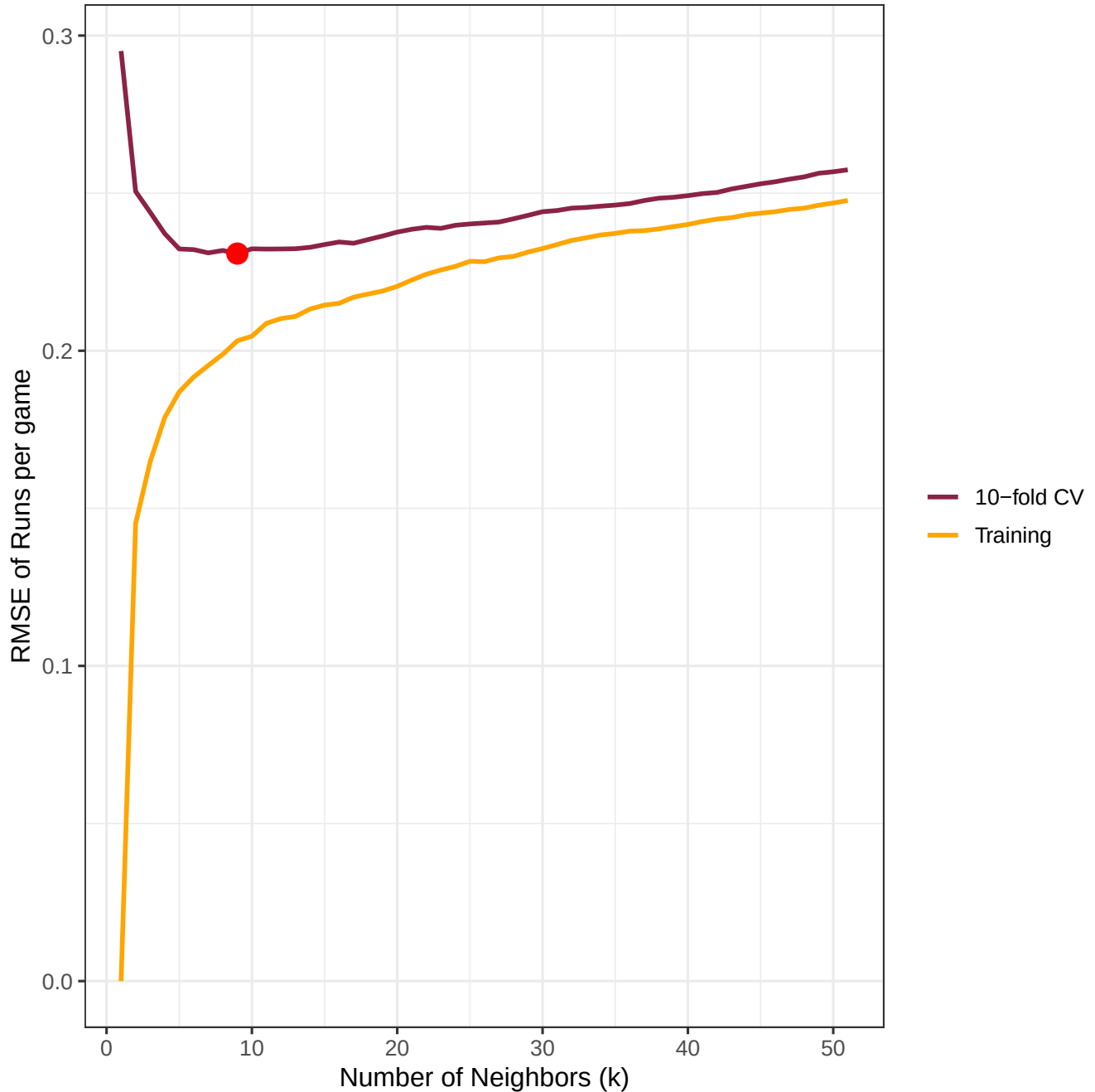


Figure 1: kNN regression tuning: cross-validated and training RMSE versus k . The red point marks the CV-optimal k .

The CV-optimal value is $k^* = 9$. Very small k gives a near-zero training RMSE since each point is nearly its own neighbor (overfitting) so the model predicts well but has high variance. The RMSE then increases since the predictions become smoother and the model has more bias (underfitting). The CV RMSE starts off higher and then decreases as the predictions become better. At $k = 9$, the model has the best possible balance of bias and variance. The following table shows the other possible k values.

Table 3: Top five candidate values of k by cross-validated RMSE. The chosen optimum is $k^* = 9$.

k	CV RMSE	Training RMSE
9	0.231	0.203
7	0.231	0.195
8	0.232	0.199
6	0.232	0.192
11	0.232	0.209

Model Performance vs the No-Skill Model

To evaluate model performance, we computed RMSE, Mean Absolute Error (MAE), and the Coefficient of Determination (R^2) on the held out test set. The metrics were compared to a null-skill baseline model.

Table 4: kNN regression on the held-out test set versus the null (mean-only) model.

Model	RMSE	MAE	R-squared
Null (training mean)	0.563	0.450	0.000
kNN ($k^* = 9$)	0.233	0.184	0.829

RMSE: Our null model was off by about half of a run every game when predicting the training average. Our $k^* = 9$ was off by about a quarter of a run every game, which is about a 50% reduction compared to our null model.

MAE: Our null models' absolute average miss is a little under half a run every game. The $k^* = 9$ model was off a little under half that, also being a little more than a 50% reduction compared to the null.

R Squared: As expected, our null model takes 0% of the predictors into account for its predictions. However, our $k^* = 9$ model takes 82%, signifying that the 10 predictors are a strong representation of the runs scored.

kNN Classification

Tuning k

We used a similar process to tune k for the classification as we did for the kNN regression. We searched odd k from 1 to $k = 1, \dots, 51$ with 10-fold CV on the training data only, using balanced accuracy and multi-class log loss to evaluate k .

Since we have multi-class targets we use the argmax rule to predict the class with the largest probability.

Table 5: Top five candidate values of k by training log-loss and balanced accuracy.

k	Train log-loss	Train bal. acc.
1	0.916	1.000
3	0.956	0.732
5	0.972	0.672
7	0.982	0.647
9	0.987	0.631

Table 6: Top five candidate values of k by cross-validated log-loss and balanced accuracy.

k	CV log-loss	CV bal. acc.
51	1.062	0.558
49	1.063	0.559
47	1.065	0.561
45	1.066	0.560
43	1.067	0.560

Cross validated log-loss is minimized at $k = 51$ and cross validated balanced accuracy is maximized at $k = 41$ the two metrics did not agree. We chose the final $k^* = 51$ using **log-loss**, because it is a strictly proper scoring rule that rewards well-calibrated probabilities and is not affected by the arbitrary argmax decision rule, which matters because we compare kNN to Naive Bayes on probability quality as well as on hard classification.

In the balanced accuracy graph, the training curve starts at a perfect 1.0 when $k = 1$ because a 1-nearest-neighbor model completely memorizes the training data. As k increases, training accuracy drops sharply and flattens out due to increased model bias. The CV curve starts low, gradually rises, and peaks at the optimal point $k = 41$ before leveling off. This indicates that $k = 41$ provides the best generalization for hard classification boundaries across unseen folds.

In the log-loss graph, the CV curve starts extremely high (around 1.34) at $k = 1$. This reflects severe overfitting and poor probability calibration, meaning when a 1-NN model is wrong on a validation fold, it is often overly confident, causing massive log-loss penalties. As k increases, the probability estimates smooth out, and log-loss steadily drops until it reaches its minimum at $k = 51$. The training curve starts low at small values of k and gradually increases as k grows, illustrating how smoothing trades away training-set precision for better generalization.

Model Performance vs the No-Skill Model

Table 7: Confusion matrix for the final kNN test set evaluation ($k^* = 51$).

	Catcher	Infield	Outfield	Pitcher
Catcher	18	30	25	11
Infield	1227	3861	2676	218
Outfield	493	1583	1800	34
Pitcher	14	25	9	87

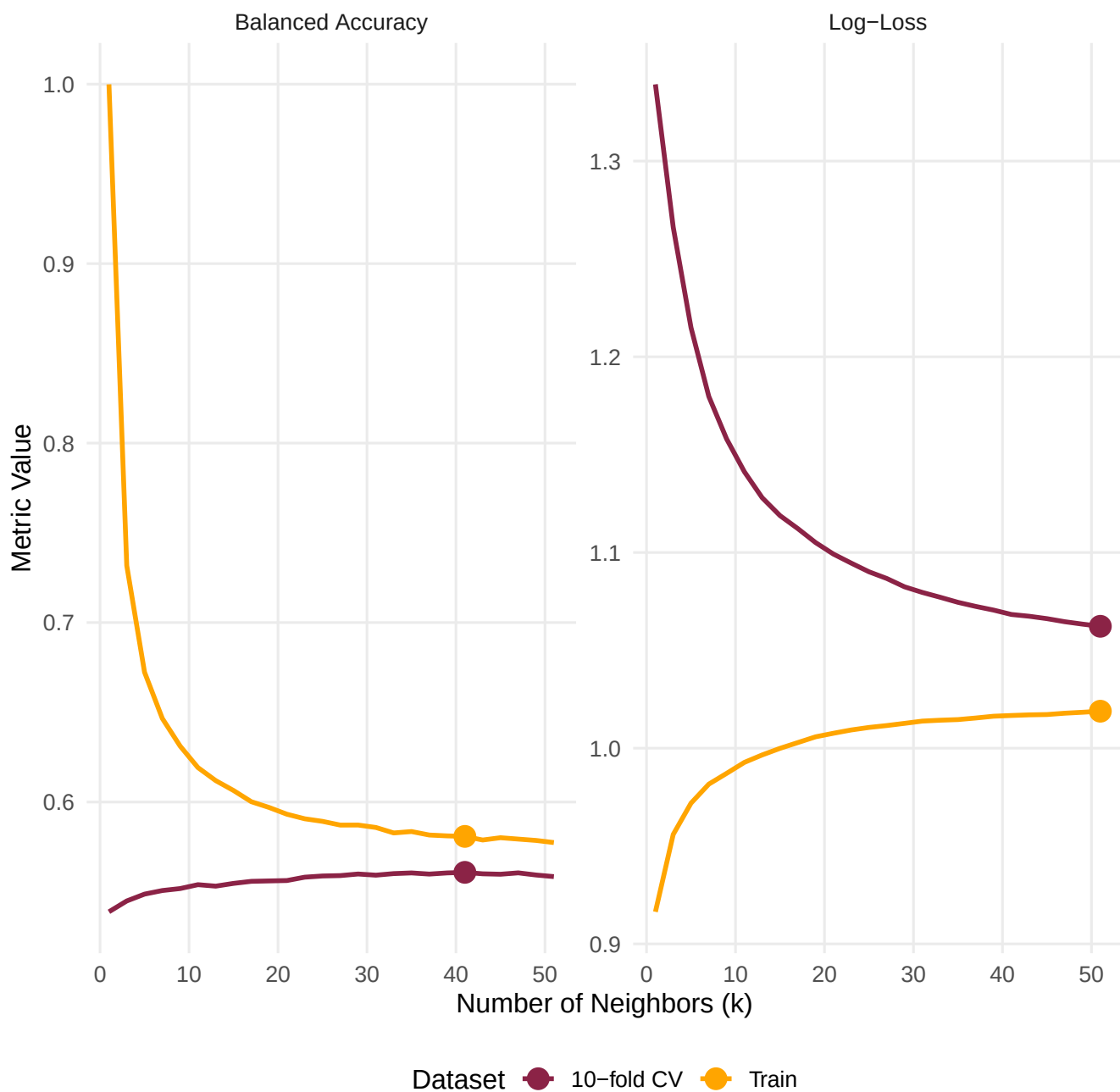


Figure 2: kNN classification tuning: cross-validated and training balanced accuracy and log-loss vs k. The dots mark the optimal k.

Table 8: Performance comparison of final kNN model ($k^* = 51$) against the no-skill baseline on the test set.

Metric	No-Skill Baseline	kNN ($k^* = 51$)
Log-Loss	1.1084	1.0638
Relative Improvement over Null	0.00%	4.03%
Brier Score	0.633389164764908	0.6182
Balanced Accuracy (Macro)	0.5	0.5561
Macro-F1	0.369035525891692	0.3447

The following metrics were chosen to evaluate model performance: log-loss since it penalizes overconfident wrong predictions; Brier score since it is threshold-free, robust measure of calibration; balanced accuracy since it balances sensitivity and specificity; and F1 since it balances precision and recall. We chose to macro-average these metrics since it ensures every class gets an equal voice, regardless of its frequency in the dataset.

The final kNN model shows modest improvements over the no-skill baseline. Log-Loss improves by 4.03% and the Brier Score decreases from 0.6334 to 0.6182, indicating better probabilistic predictions. Balanced Accuracy also increases from 0.50 to 0.5561, suggesting better performance across classes. However, Macro-F1 decreases from 0.3690 to 0.3447, indicating that the model does not improve the overall precision-recall balance across classes. Overall, kNN provides better probability estimates and balanced accuracy, but its classification performance as measured by Macro-F1 remains weaker than the no-skill baseline.

Naive Bayes Classification

Priors

Table 9: Training-set class counts and proportions (π_k).

	Catcher	Infield	Outfield	Pitcher
Count	4087.000	12832.000	10523.000	817.000
Proportion	0.145	0.454	0.372	0.029

Naive Bayes uses these class proportions as its prior probabilities. The most common position is the Infield position.

Fitting the Model

Naive Bayes assumes each predictor is independent and follows a Gaussian distribution. It has no hyperparameter to tune, so there is no cross-validation step. We fit it on the same training data used for kNN and evaluated it once on the test set, using the same metrics as for kNN.

Table 10: Confusion matrix for Naive Bayes on the test set.

	Catcher	Infield	Outfield	Pitcher
Catcher	8	12	13	0
Infield	1278	4124	3012	215
Outfield	407	1278	1446	13
Pitcher	59	85	39	122

Model Performance vs the Null Skill Model

Table 11: Confusion matrix for the no-skill model on the test set (always predicts the majority class).

	Catcher	Infield	Outfield	Pitcher
Catcher	0	0	0	0
Infield	1752	5499	4510	350
Outfield	0	0	0	0
Pitcher	0	0	0	0

Table 12: Performance comparison of Naive Bayes against the no-skill baseline on the test set.

Metric	No-Skill Baseline	Naive Bayes
Log-Loss	1.1084	1.0689
Relative Improvement over Null	0.00%	3.57%
Brier Score	0.6334	0.6218
Balanced Accuracy (Macro)	0.5	0.5626
Macro-F1	0.369	0.3358

Naive Bayes performs slightly better than the no-skill baseline overall, but the improvement is modest. The Log-Loss decreases from 1.1084 to 1.0689, a 3.57% improvement, meaning Naive Bayes produces somewhat better-calibrated probability predictions. The Brier Score decreases from 0.6334 to 0.6218, also indicating slightly better probabilistic predictions. Balanced Accuracy increases from 0.50 to 0.5626, so Naive Bayes is better at correctly identifying classes while accounting for class imbalance. Macro-F1 actually decreases from 0.369 to 0.3358. This suggests that although the model improves balanced accuracy, its precision/recall tradeoff across classes is not as good as the baseline.

Overall, Naive Bayes provides a small improvement in probabilistic performance and balanced classification accuracy, but the lower Macro-F1 shows that it does not consistently improve classification performance across all classes.

Comparing kNN vs Naive Bayes Classification

Table 13: Test-set comparison of the no-skill baseline, kNN, and Naive Bayes.

Metric	No-Skill	kNN ($k^* = 51$)	Naive Bayes
Log-Loss	1.108	1.064	1.069
Brier Score	0.633	0.618	0.622
Balanced Accuracy (Macro)	0.500	0.556	0.563
Macro-F1	0.369	0.345	0.336

Overall, both kNN and Naive Bayes outperform the no-skill baseline, but they have different strengths. kNN has slightly better Log-Loss (1.064) and Brier Score (0.618), indicating better probabilistic predictions. However, Naive Bayes achieves the highest Balanced Accuracy (0.563) compared with kNN (0.556), suggesting it performs slightly better across classes when accounting for class imbalance. Both models have lower Macro-F1 than the baseline, with Naive Bayes performing lowest (0.336). Thus, kNN is slightly better for probability-based performance, while Naive Bayes has a small advantage in balanced accuracy, and neither clearly improves Macro-F1.

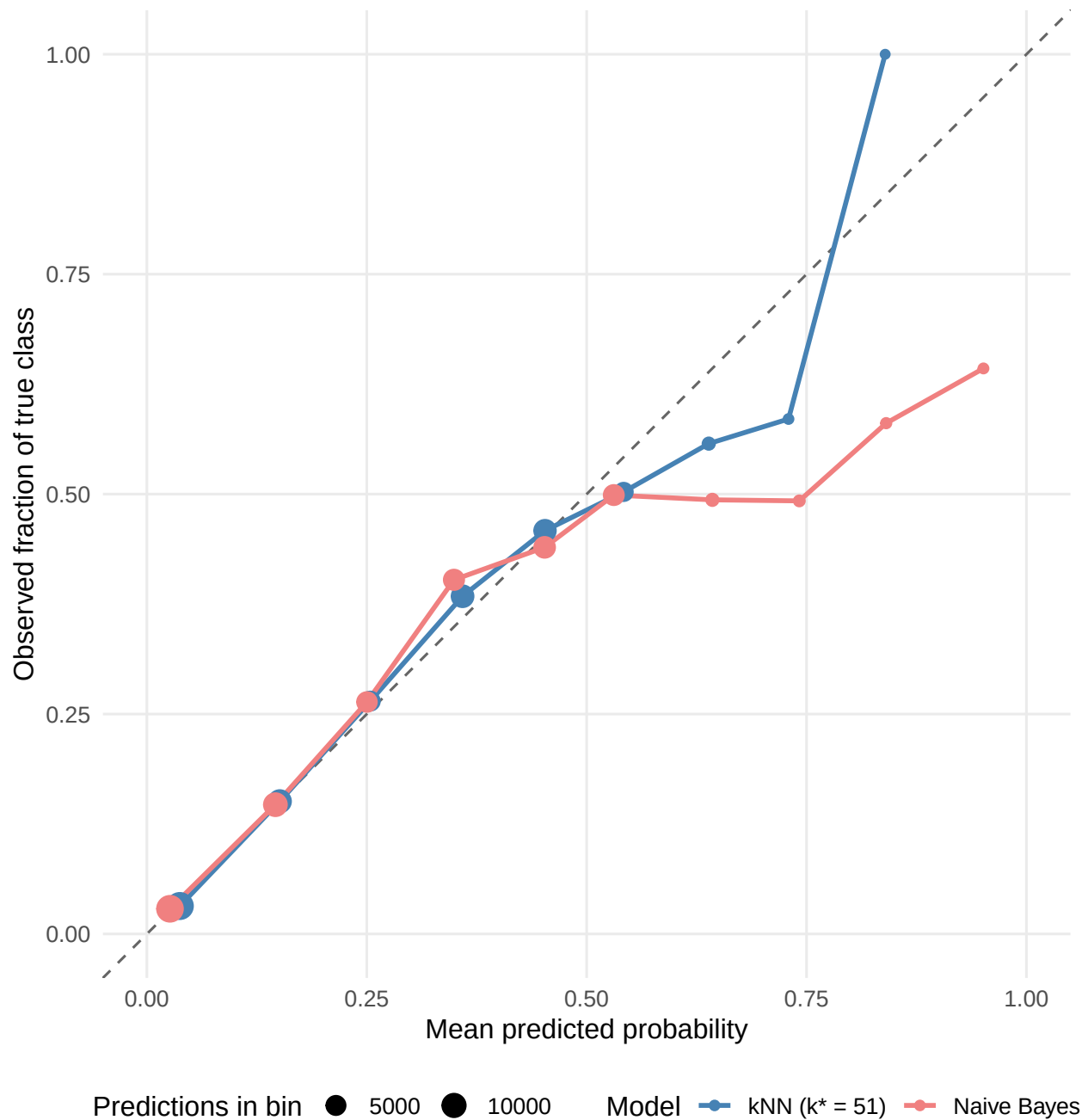


Figure 3: Calibration plot comparing kNN and Naive Bayes Classification. Diagonal line represents perfect calibration.

In the low to mid probability range, Both models track very closely along the diagonal line, indicating that their probability estimates are well-calibrated and reliable when predicting lower or moderate confidence scores. The point sizes show high data density in this lower range.

In the high probability range (greater than 0.5), the kNN model falls below the diagonal in the mid-high range, but shoots up sharply at the highest probability bins. This suggests some degree of overconfidence or data sparsity at the upper extremes. The kNN model hugs the diagonal more closely than the Naive Bayes Model, which flattens out at the higher probabilities. This is because of the independence assumption; the probabilities become overconfident.

5. Discussion and Real World Implications

Accuracy and log-loss

Both our kNN and Naive Bayes model had better balanced accuracy and log-log loss than our null prediction. A comparison to note is that while Naive Bayes had a slightly higher balanced accuracy than kNN, it also contain a slightly higher and worse log-loss score. This is explained by the nature of the two models in comparison to our data. Naive Bayes assumes independence no matter what, which also increases the confidence of its predictions in comparison to kNN. This is an issue mainly because our data is highly correlated. However, in contrast, the independence assumption prevents other less common positions such as pitcher from receiving influenced predictions, improving the accuracy.

Generative vs. Non-Parametric Trade-Offs

As mentioned earlier, offensive stats in baseball are highly correlated. One of the outcomes are going to happen in any and every at bat. This fact caused the Naive Bayes's independence assumption to make overconfident predictions, making those that fail receive harsher penalties on our probability scores like log-loss and Briers. In terms of accuracy, however, the independence assumption did not have any negative effect. When comparing the two models, their metrics are still close enough that a different split could have effected switched these values.

kNN Neighborhood Size

In both our classification and Regression we can see the overfitting at the small $k = 1$. As we approach our chosen k , see a sharp decline in our RMSE and log-loss, along with an incline in our balanced accuracy. However, as we go past our k and begin underfitting, we start seeing these values going in the opposite direction, and eventually going past the null prediction's scores.

Real-World decision

In the context of the sports domain, a false negative can be more dangerous. One example would be scouting. A player in a needed position (Catcher) is flagged a under performing, which can cause a scout to overlook said player. In this situation, both false-positives and false-negatives can be easily checked, but still have some importance. Practitioners will want to look at sensitivity over specificity in this case, as you want to lower false negatives. This also means we want our threshold to be lower, increasing the false positives w get while lowering false negatives.

6. Citations and Data Link

Lahman Baseball Database – Society for American Baseball Research. (2025). Retrieved September 29, 2026, from <https://sabr.org/lahman-database/>

The data is linked [here](#).

7. Appendix

Data Preprocessing

```
pos      <- read.csv("Fielding.csv")
batting  <- read.csv("batting.csv")
team     <- read.csv("Teams.csv")
# Getting highest position played for players
primary_pos <- pos %>%
  group_by(playerID, yearID, POS) %>%
  summarise(G = sum(G), .groups = "drop") %>%
  group_by(playerID, yearID) %>%
  slice_max(G, n = 1, with_ties = FALSE) %>%
  select(playerID, yearID, POS)

# Merge Position into Player and Year to condense data
stats <- merge(batting, primary_pos,
               by = c("playerID", "yearID"), all.x = TRUE)

# Keeps only American and National League
stats_clean <- stats[stats$lgID %in% c("AL", "NL"), ]
stats_clean <- stats_clean[!is.na(stats_clean$POS), ]

##### REGRESSION
## Separates by team for regression prediction.

team_clean <- team[team$lgID %in% c("AL", "NL"),] # Keeps only AL and NL
team_clean$X1B <- team_clean$H - team_clean$X2B - team_clean$X3B - team_clean$HR # Creates X1B
team_clean$HBPxBB <- team_clean$BB + team_clean$HBP # Creates HBPxBB

# Takes only the necessary data
team_clean <- team_clean[,c("yearID", "teamID", "G", "R", "X1B", "X2B", "X3B",
                           "HR", "SO", "HBPxBB", "SB", "SF", "CS", "AB")]

#Divides stats by games to establish per game base
stats <- c("R", "X1B", "X2B", "X3B", "HR", "SO", "HBPxBB", "SB", "SF", "CS", "AB")
team_clean[stats] <- team_clean[stats] / team_clean$G

team_clean <- team_clean[!is.na(team_clean$SF), ]# Removes NaN values from SF
team_clean <- team_clean[!is.na(team_clean$HBPxBB), ]# Removes NaN values from HBPxBB

# List of predictors for training
num_vars <- c("X1B", "X2B", "X3B", "HR", "SO", "HBPxBB", "SB", "SF", "CS", "AB")

##### CLASSIFICATION AND BAYES
# Keeps only At Bats > 100
pos_data <- stats_clean[stats_clean$AB >= 100, ]

# Changes The infield positions into a common infield tab
for(i in 1:length(pos_data$POS)){
```

```

if(pos_data$POS[i] == "1B" ||
   pos_data$POS[i] == "2B" ||
   pos_data$POS[i] == "3B" ||
   pos_data$POS[i] == "SS" ){
  pos_data$POS[i] = "IF"
}
}
# For easier referral
position_names <- c(
  "IF" = "Infield",
  "C"  = "Catcher",
  "OF" = "Outfield",
  "P"  = "Pitcher"
)

#Factors Positions into levels
pos_data$Position <- factor(position_names[pos_data$POS])

#Keep only necessary data
pos_data <- pos_data[, c("playerID", "yearID", "Position",
                        "H", "HR", "BB", "SO", "SB", "AB")]

# creates scaled pos_data for Classification comparison and naive Bayes
pos_scale <- pos_data

pos_scale$BA <- pos_data$H / pos_data$AB
pos_scale$HRA <- pos_data$HR / pos_data$AB
pos_scale$BBA <- pos_data$BB / pos_data$AB
pos_scale$SOA <- pos_data$SO / pos_data$AB

#Adds scaled variables into pos_scaled
pos_scale <- pos_scale[, c("playerID", "yearID", "Position",
                          "BA", "HRA", "BBA", "SOA")]

# List of Predictors for pos_scaled
feats <- c("BA", "HRA", "BBA", "SOA")

# table formatting function
tbl <- function(x, cap, digits = 3, rn = FALSE, ...) {
  knitr::kable(x, booktabs = TRUE, caption = cap, digits = digits,
               position = "H", row.names = rn, ...)
}

d2 <- unlist((team_clean[1, num_vars] - team_clean[2, num_vars])^2)
z <- scale(team_clean[, num_vars])
d2z <- (z[1, ] - z[2, ])^2
demo <- rbind("Raw (share of squared distance, pct)" = 100 * d2 / sum(d2),
             "Z-scored (share of squared distance, pct)" = 100 * d2z / sum(d2z))
tbl(demo, "Share of squared Euclidean distance contributed by each predictor (first two games) in the kNN Regr",
    digits = 1, rn = TRUE)

d2 <- unlist((pos_scale[1, feats] - pos_scale[2, feats])^2)
z <- scale(pos_scale[, feats])
d2z <- (z[1, ] - z[2, ])^2
demo <- rbind("Raw (share of squared distance, pct)" = 100 * d2 / sum(d2),
             "Z-scored (share of squared distance, pct)" = 100 * d2z / sum(d2z))
tbl(demo, "Share of squared Euclidean distance contributed by each predictor (first two team-seasons) in the C",
    digits = 1, rn = TRUE)

```

```

#regression split 70/30
set.seed(4654)
idx_reg <- sample(nrow(team_clean), round(0.7 * nrow(team_clean)))
train_raw <- team_clean[idx_reg, num_vars]
test_raw <- team_clean[-idx_reg, num_vars]
train_y <- team_clean$R[idx_reg]
test_y <- team_clean$R[-idx_reg]

mu_reg <- colMeans(train_raw)
sd_reg <- apply(train_raw, 2, sd)
training_df <- scale(train_raw, center = mu_reg, scale = sd_reg)
testing_df <- scale(test_raw, center = mu_reg, scale = sd_reg)

# classification split (knn and nb)

set.seed(4654)
idx_cls <- unlist(lapply(split(seq_len(nrow(pos_scale))), pos_scale$Position),
                  function(i) sample(i, round(0.7 * length(i)))))
cls_train <- pos_scale[idx_cls, ]
cls_test <- pos_scale[-idx_cls, ]

mu_cls <- colMeans(cls_train[, feats])
sd_cls <- apply(cls_train[, feats], 2, sd)
train_x <- scale(cls_train[, feats], center = mu_cls, scale = sd_cls)
test_x <- scale(cls_test[, feats], center = mu_cls, scale = sd_cls)

lv <- levels(pos_scale$Position)
K <- length(lv)
y_tr <- as.integer(cls_train$Position)
y_te <- as.integer(cls_test$Position)

# null entropy log loss baseline for knn and nb classification
pi_train <- prop.table(table(cls_train$Position))[lv]
null_entropy <- -sum(pi_train * log(pi_train))

```

kNN Regression

```

set.seed(4654)
folds <- sample(rep(1:10, length.out = nrow(train_raw)))

# creates folds, scaling inside folds
fold_data <- lapply(1:10, function(f) {
  tr <- train_raw[folds != f, ] #training fold
  va <- train_raw[folds == f, ] #validation fold
  m <- colMeans(tr)
  s <- apply(tr, 2, sd)
  list(tr_x = scale(tr, center = m, scale = s), #response
       va_x = scale(va, center = m, scale = s),
       tr_y = train_y[folds != f], #predictor
       va_y = train_y[folds == f])
})

# function to train knn on inputted val of k on the 10 folds created with fold_data()
cv_rmse <- function(k) {
  sq_err <- unlist(lapply(fold_data, function(d) {
    pred <- knn.reg(train = d$tr_x, test = d$va_x, y = d$tr_y, k = k)$pred
    (d$va_y - pred)^2
  })))
  sqrt(mean(sq_err))
}

```

```

}

# train k without CV
train_rmse <- function(k) {
  pred <- knn.reg(train = training_df, test = training_df, y = train_y, k = k)$pred
  sqrt(mean((train_y - pred)^2))
}

k_cv      <- 1:51
cv_results <- data.frame(k = k_cv,
                        RMSE = sapply(k_cv, cv_rmse),
                        Train_RMSE = sapply(k_cv, train_rmse))
k_best <- cv_results$k[which.min(cv_results$RMSE)]

ggplot(cv_results, aes(x = k)) +
  geom_line(aes(y = RMSE, color = "10-fold CV"), linewidth = 1) +
  geom_line(aes(y = Train_RMSE, color = "Training"), linewidth = 1) +
  geom_point(data = cv_results[k == k_best, ],
            aes(y = RMSE), color = "red", size = 4) +
  scale_color_manual(values = c("10-fold CV" = "#8B2346", "Training" = "orange")) +
  labs(
    x = "Number of Neighbors (k)", y = "RMSE of Runs per game", color = NULL) +
  theme_bw(base_size = 12)

top_k <- head(cv_results[order(cv_results$RMSE), c("k", "RMSE", "Train_RMSE")], 5)
names(top_k) <- c("k", "CV RMSE", "Training RMSE")
tbl(top_k, paste0("Top five candidate values of k by cross-validated RMSE. The chosen optimum is k* = ",
                  k_best, "."))

# final model
final_pred <- knn.reg(train = training_df, test = testing_df, y = train_y, k = k_best)$pred

rmse      <- function(a, b) sqrt(mean((a - b)^2))
mae       <- function(a, b) mean(abs(a - b))
r_squared <- function(a, b) 1 - sum((a - b)^2) / sum((a - mean(a))^2)

null_prediction <- rep(mean(train_y), length(test_y))

results <- data.frame(
  Model = c("Null (training mean)", paste0("kNN (k* = ", k_best, ")")),
  RMSE  = c(rmse(test_y, null_prediction), rmse(test_y, final_pred)),
  MAE   = c(mae(test_y, null_prediction), mae(test_y, final_pred)),
  R2    = c(r_squared(test_y, null_prediction), r_squared(test_y, final_pred))
)

results[, -1] <- round(results[, -1], 3)

names(results)[4] <- "R-squared"
tbl(results, "kNN regression on the held-out test set versus the null (mean-only) model.")

```

kNN Classification

```

# helper to convert classes with 1 hot encoding
onehot <- function(y_int) diag(K)[y_int, , drop = FALSE]
# calc log loss
logloss <- function(P, y_int) -mean(log(pmax(P[cbind(seq_along(y_int), y_int)], 1e-15)))
# calc brier score
brier <- function(P, y_int) mean(rowSums((P - onehot(y_int))^2))
#argmax

```

```

pred_from_P <- function(P) max.col(P, ties.method = "first")

# create confusion matrix
conf_tab <- function(pred_int, y_int) {
  table(Predicted = factor(pred_int, levels = 1:K, labels = lv),
        Actual    = factor(y_int,   levels = 1:K, labels = lv))
}

# metrics per class, input confusion matrix
per_class <- function(cm) {
  TP <- diag(cm); FP <- rowSums(cm) - TP; FN <- colSums(cm) - TP
  TN <- sum(cm) - TP - FP - FN
  sens <- TP / (TP + FN); spec <- TN / (TN + FP)
  data.frame(
    Sensitivity = sens,
    Specificity = spec,
    F1          = 2 * TP / (2 * TP + FP + FN),
    BalAcc      = (sens + spec) / 2
  )
}

# summary to evaluate model performance
summ_metrics <- function(P, y_int) {
  cm <- conf_tab(pred_from_P(P), y_int)
  pc <- per_class(cm)
  b  <- brier(P, y_int)

  c(LogLoss = logloss(P, y_int),
    Brier    = b,
    RMSE     = sqrt(b),
    BalAcc   = mean(pc$BalAcc, na.rm = TRUE),
    MacroF1  = mean(pc$F1, na.rm = TRUE))
}

# knn grid
k_grid <- seq(1, 51, by = 2)
kmax   <- max(k_grid)

# neighbor labeks
nbr_labels <- function(train_mat, query_mat, y_int, kmax) {
  nn <- FNN::knnx.index(train_mat, query_mat, k = kmax)
  matrix(y_int[nn], nrow = nrow(nn))
}

# smoothed vote fractions
prob_k <- function(NL, k) {
  votes <- sapply(seq_len(K), function(cl) rowSums(NL[, seq_len(k), drop = FALSE] == cl))
  P <- (votes + 1) / (k + K)
  colnames(P) <- lv
  P
}

# 10-fold CV loop
set.seed(4654)
folds_c <- sample(rep(1:10, length.out = nrow(cls_train)))
NL_cv   <- matrix(NA_integer_, nrow(cls_train), kmax)

for (f in 1:10) {
  tr <- which(folds_c != f)

```

```

va <- which(folds_c == f)
m <- colMeans(cls_train[tr, feats])
s <- apply(cls_train[tr, feats], 2, sd)
NL_cv[va, ] <- nbr_labels(
  scale(cls_train[tr, feats], center = m, scale = s),
  scale(cls_train[va, feats], center = m, scale = s),
  y_tr[tr], kmax
)
}

NL_train <- nbr_labels(train_x, train_x, y_tr, kmax)

# tuning
tune <- do.call(rbind, lapply(k_grid, function(k) {
  Pcv <- prob_k(NL_cv, k);   cvm <- summ_metrics(Pcv, y_tr)
  Ptr <- prob_k(NL_train, k); trm <- summ_metrics(Ptr, y_tr)

  data.frame(
    k          = k,
    CV_LogLoss = unname(cvm["LogLoss"]),
    Train_LogLoss = unname(trm["LogLoss"]),
    CV_BalAcc  = unname(cvm["BalAcc"]),
    Train_BalAcc = unname(trm["BalAcc"]),
    CV_Brier   = unname(cvm["Brier"]),
    CV_RMSE    = unname(cvm["RMSE"]),
    CV_MacroF1 = unname(cvm["MacroF1"])
  )
}))

# optimal k choices based on the two evaluation metrics
k_ll <- tune$k[which.min(tune$CV_LogLoss)]
k_ba <- tune$k[which.max(tune$CV_BalAcc)]
k_star = k_ll #final choice is log loss

top_k_cv <- head(tune[order(tune$CV_LogLoss), c("k", "CV_LogLoss", "CV_BalAcc")], 5)
top_k_tr <- head(tune[order(tune$Train_LogLoss), c("k", "Train_LogLoss", "Train_BalAcc")], 5)

names(top_k_cv) <- c("k", "CV log-loss", "CV bal. acc.")
names(top_k_tr) <- c("k", "Train log-loss", "Train bal. acc.")

tbl(top_k_tr, "Top five candidate values of k by training log-loss and balanced accuracy.")
tbl(top_k_cv, "Top five candidate values of k by cross-validated log-loss and balanced accuracy.")

# reshape tune data into long format for faceting
tune_long <- data.frame(
  k      = rep(tune$k, 4),
  Metric = rep(c("Log-Loss", "Log-Loss", "Balanced Accuracy", "Balanced Accuracy"), each = nrow(tune)),
  Set     = rep(c("10-fold CV", "Train", "10-fold CV", "Train"), each = nrow(tune)),
  Value   = c(tune$CV_LogLoss, tune$Train_LogLoss, tune$CV_BalAcc, tune$Train_BalAcc)
)

# optimal k values
opt_points <- data.frame(
  k      = c(k_ll, k_ll, k_ba, k_ba),
  Metric = c("Log-Loss", "Log-Loss", "Balanced Accuracy", "Balanced Accuracy"),
  Set     = c("10-fold CV", "Train", "10-fold CV", "Train"),
  Value   = c(
    tune$CV_LogLoss[tune$k == k_ll],
    tune$Train_LogLoss[tune$k == k_ll],

```

```

    tune$CV_BalAcc[tune$k == k_ba],
    tune$Train_BalAcc[tune$k == k_ba]
  )
)

# plot
ggplot(tune_long, aes(x = k, y = Value, color = Set)) +
  geom_line(linewidth = 1) +
  facet_wrap(~ Metric, scales = "free_y", nrow = 1) +
  geom_point(data = opt_points, aes(x = k, y = Value, color = Set), size = 4, inherit.aes = FALSE) +
  scale_color_manual(values = c("10-fold CV" = "#8B2346", "Train" = "orange")) +
  labs(
    x = "Number of Neighbors (k)",
    y = "Metric Value",
    color = "Dataset"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "bottom",
    panel.grid.minor = element_blank()
  )

# Generate neighbor labels for the test set using kmax, then subset for k_star
NL_test <- nbr_labels(train_x, test_x, y_tr, kmax)
P_test <- prob_k(NL_test, k_star)

# Compute test metrics
test_metrics <- summ_metrics(P_test, y_te)

# relative improvement
rel_improvement <- 1 - (test_metrics["LogLoss"] / null_entropy)

# confusion matrix
test_pred <- pred_from_P(P_test)
test_cm <- conf_tab(test_pred, y_te)
tbl(as.data.frame.matrix(test_cm),
    paste0("Confusion matrix for the final kNN test set evaluation (k* = ", k_star, ")."), rn=TRUE)

# comparison table
comparison_df <- data.frame(
  Metric = c("Log-Loss", "Relative Improvement over Null", "Brier Score", "Balanced Accuracy (Macro)", "Macro-
  No_Skill_Baseline = c(round(null_entropy, 4), "0.00%", 1 - sum(pi_train^2), "0.5", mean(2 * pi_train / (1 +
  Final_Model_k_star = c(
    round(test_metrics["LogLoss"], 4),
    paste0(round(rel_improvement * 100, 2), "%"),
    round(test_metrics["Brier"], 4),
    round(test_metrics["BalAcc"], 4),
    round(test_metrics["MacroF1"], 4)
  )
)
names(comparison_df) <- c("Metric", "No-Skill Baseline", paste0("kNN (k* = ", k_star, ")"))
tbl(comparison_df, paste0("Performance comparison of final kNN model (k* = ", k_star, ") against the no-skill

```

Naive Bayes Classification

```

train_counts <- as.integer(table(cls_train$Position)[lv])
pi_tab <- rbind(Count = train_counts,
  Proportion = round(as.numeric(pi_train), 4))
colnames(pi_tab) <- lv

```

```

tbl(pi_tab, "Training-set class counts and proportions ($\\pi_k$).",
    rn = TRUE)

fitNB <- naiveBayes(Position ~ ., data = data.frame(Position = cls_train$Position, train_x))

# test-set class probabilities
P_nb <- predict(fitNB, newdata = as.data.frame(test_x), type = "raw")[, lv, drop = FALSE]

# confusion matrix
nb_metrics <- summ_metrics(P_nb, y_te)
nb_cm <- conf_tab(pred_from_P(P_nb), y_te)

tbl(as.data.frame.matrix(nb_cm),
    "Confusion matrix for Naive Bayes on the test set.", rn = TRUE)

# no-skill baselines from the training priors
pi_nb <- as.numeric(prop.table(table(factor(lv[y_tr], levels = lv))))
entropy_nb <- -sum(pi_nb * log(pi_nb))

null_base <- c(
  LogLoss = entropy_nb,
  Brier = 1 - sum(pi_nb^2),
  BalAcc = 0.5,
  MacroF1 = mean(2 * pi_nb / (1 + pi_nb))
)

# null model confusion matrix
maj_int <- which.max(table(factor(y_tr, levels = 1:K)))
null_pred <- rep(maj_int, length(y_te))
null_cm <- conf_tab(null_pred, y_te)

tbl(as.data.frame.matrix(null_cm),
    "Confusion matrix for the no-skill model on the test set (always predicts the majority class).",
    rn = TRUE)

# relative log-loss improvement over no-skill
rel_improvement_nb <- 1 - nb_metrics["LogLoss"] / null_base["LogLoss"]

comparison_nb <- data.frame(
  Metric = c("Log-Loss", "Relative Improvement over Null", "Brier Score",
    "Balanced Accuracy (Macro)", "Macro-F1"),
  NoSkill = c(round(null_base["LogLoss"], 4),
    "0.00%",
    round(null_base["Brier"], 4),
    round(null_base["BalAcc"], 4),
    round(null_base["MacroF1"], 4)),
  NB = c(round(nb_metrics["LogLoss"], 4),
    paste0(round(rel_improvement_nb * 100, 2), "%"),
    round(nb_metrics["Brier"], 4),
    round(nb_metrics["BalAcc"], 4),
    round(nb_metrics["MacroF1"], 4))
)
names(comparison_nb) <- c("Metric", "No-Skill Baseline", "Naive Bayes")
tbl(comparison_nb, "Performance comparison of Naive Bayes against the no-skill baseline on the test set.")

```

Comparison of Classifications

```

final_comp <- data.frame(
  Metric = c("Log-Loss", "Brier Score", "Balanced Accuracy (Macro)", "Macro-F1"),
  NoSkill = round(null_base[c("LogLoss", "Brier", "BalAcc", "MacroF1")], 4),
  kNN = round(test_metrics[c("LogLoss", "Brier", "BalAcc", "MacroF1")], 4),
  NB = round(nb_metrics[c("LogLoss", "Brier", "BalAcc", "MacroF1")], 4)
)
names(final_comp) <- c("Metric", "No-Skill", paste0("kNN (k* = ", k_star, ")"), "Naive Bayes")
tbl(final_comp, "Test-set comparison of the no-skill baseline, kNN, and Naive Bayes.")

calib_df <- function(P, y_int, model, bins = 10) {
  df <- data.frame(p = as.vector(P), # every class-probability
                  obs = as.vector(onehot(y_int))) # 1 if that class was the true one
  df$bin <- cut(df$p, breaks = seq(0, 1, length.out = bins + 1), include.lowest = TRUE)
  out <- do.call(rbind, lapply(split(df, droplevels(df$bin)), function(d) {
    data.frame(mean_p = mean(d$p), frac_obs = mean(d$obs), n = nrow(d))
  }))
  out$Model <- model
  out
}

calib <- rbind(
  calib_df(P_test, y_te, paste0("kNN (k* = ", k_star, ")")),
  calib_df(P_nb, y_te, "Naive Bayes")
)

ggplot(calib, aes(x = mean_p, y = frac_obs, color = Model)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey40") +
  geom_line(linewidth = 1) +
  geom_point(aes(size = n)) +
  scale_color_manual(values = c("steelblue", "lightcoral")) +
  scale_size_continuous(range = c(1.5, 5), name = "Predictions in bin") +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(x = "Mean predicted probability",
       y = "Observed fraction of true class") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom", panel.grid.minor = element_blank())

```